ML-Based Python Code Summarization using Transformer Architecture

Muhammad Sohail (7178621)
Syed Umair Anwar (7172248)

May 9, 2026

Contents

1. Project Overview

This project implements a machine learning-based Python code summarization system using a sequence-to-sequence Transformer architecture. The goal is to automatically generate natural language summaries from Python source code, creating meaningful docstrings that describe functionality. The system employs a Transformer-based encoder-decoder architecture trained on Python code and corresponding documentation pairs. The model learns to map source code tokens to natural language summaries, capturing semantic relationships between code structure and its functionality.

2. Project Structure

The project is organized into logical modules separating model definition, data handling, and execution scripts:

```
project_root/
+-- data/
|   +-- tokenizer.json              # trained BPE tokenizer
|   +-- tokenizer_training.ipynb    # tokenizer training notebook
+-- models/
|   +-- transformer.py    # encoder-decoder transformer
+-- utils/
|   +-- dataset.py        # dataset class and preprocessing
+-- notebooks/
|   +-- training.ipynb    # training visualization
|   +-- evaluation.ipynb  # evaluation visualization
+-- train.py              # training
+-- evaluate.py           # metrics
+-- summarize.py          # demo
+-- report/report.pdf     # report
```

- `models/transformer.py` - encoder-decoder Transformer with multi-head self-attention
- `utils/dataset.py` - PyTorch `Dataset` interface for loading and tokenizing code-docstring pairs
- `train.py` - training loop with optimizer, loss computation, and checkpointing
- `evaluate.py` - evaluation metrics on held-out test data
- `summarize.py` - CLI for generating summaries from new code snippets
- `data/` - preprocessed data and trained artifacts for reproducibility

3. Data Source Selection

We used the **CodeXGLUE** [2] benchmark (General Language Understanding Evaluation for Code). CodeXGLUE is a benchmark suite that incorporates several datasets, including **CodeSearchNet**, to evaluate code intelligence tasks. CodeSearchNet itself is a dataset of code-documentation pairs collected from open-source GitHub repositories. CodeXGLUE wraps CodeSearchNet into a standardized format with predefined train/validation/test splits, so we get clean code-docstring pairs ready for training without having to build our own preprocessing pipeline.
We applied a Python filter to only get the code summarization portion:

```python
from datasets import load_dataset

dataset = load_dataset("google/code_x_glue_ct_code_to_text", "python")
```

This gives us train, validation, and test splits. Each example has a `code` field and a `docstring` field, which is the input-output pair our model learns from.

4. Tokenization

A tokenizer is in charge of preparing the inputs for a model. Tokenizing means splitting strings into sub-word token strings, converting tokens strings to IDs and back, and encoding/decoding (i.e., tokenizing and converting to integers). We built our tokenizer in four steps [4, 5].

Step 1 – The Shell. We initialize a tokenizer that uses BPE and has a safety net:

```
tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
```

This line initiates a shell tokenizer. It has the BPE algorithm and a safety net (`[UNK]`). We used `[UNK]` so that it does not crash on words it does not understand and puts `[UNK]` instead.

BPE (Byte-Pair Encoding) is an iterative and greedy algorithm. It starts by seeing every character individually, then it creates a merge rule pairing the most frequent characters together. After enough merges, common Python keywords like `def`, `return`, and `if` end up being treated as single, atomic tokens.

Step 2 – The Scissors (Pre-tokenizer). If the tokenizer is the shell, then `ByteLevel` is the scissors. The pre-tokenizer cuts a long, continuous string of Python code into individual initial chunks based on spaces, punctuation, and byte-values. We set `add_prefix_space=False` to avoid adding a space at the start of every string:

```
tokenizer.pre_tokenizer = ByteLevel(add_prefix_space=False)
```

We used `ByteLevel` instead of `Whitespace` because `Whitespace` would remove whitespace characters entirely. With `ByteLevel`, whitespaces get converted into bytes first, so they are preserved rather than being thrown away.

Step 3 – Trainer Configuration.

```
trainer = BpeTrainer(vocab_size=32000,
                     min_frequency=2,
                     special_tokens=["[UNK]", "[PAD]", "[SOS]", "[EOS]"])
```

`vocab_size` directly affects two parts of the model: the Embedding Layer (the input) and the Softmax Layer (the output). For example, if the vocabulary size is 5,000 and the embedding dimension is 512, the embedding matrix inside the model is a table of size $5{,}000 \times 512$. We went with 32,000 which falls in the optimal range of 25,000–32,000. At this size, most Python keywords and common variable names become single tokens. If we used something too small like 5,000, it would start splitting words up. For instance, `return` would become `["ret", "urn"]`.

`min_frequency` is set to 2, meaning a character combination has to appear at least twice in the training data before BPE will merge it into a single token.

For the special tokens, their order matters because it determines their IDs: `[UNK]`=0 is the safety net, so if a word is not in the 32k vocabulary, it gets replaced with `[UNK]` instead of crashing. `[PAD]`=1 is the padding token, used to fill extra space when one Python function is shorter than another in the same batch. `[SOS]`=2 (Start of Sequence) is the green signal that tells the model to start generating. `[EOS]`=3 (End of Sequence) tells the model when to stop. Without it, the model would never know when to stop writing.

Step 4 – Training the Tokenizer. We feed both the code and the docstring from the training split so the tokenizer learns the vocabulary of both Python code (e.g. `def`, `return`) and English text (e.g. "This function"). To avoid loading everything into memory at once, we use a generator function (`get_corpus`) that yields one string at a time:

```
tokenizer.train_from_iterator(get_corpus(), trainer=trainer)
```

We also set up a `ByteLevelDecoder` so that when we decode token IDs back into text, the byte-level characters (like `Ġ` which replaces spaces) get converted back to normal readable text. After training, the tokenizer is saved as `tokenizer.json` in the `data/` directory so it can be reloaded later for training and inference without retraining.

5. Data Loading and Preprocessing

Once the tokenizer is trained and saved, the next step is building a data pipeline that feeds code-docstring pairs into the model in the right format. This is handled by `utils/dataset.py`, which implements PyTorch's `Dataset` interface.

5.1. Special Token IDs

At the top of the file we define the special token IDs. These must match the exact order we used when training the tokenizer in Section 4, otherwise the model would confuse padding with end-of-sequence or start-of-sequence signals:

```
UNK_ID = 0
PAD_ID = 1
SOS_ID = 2
EOS_ID = 3
```

5.2. The Dataset Class

We subclass PyTorch's `Dataset`, which requires two methods: `__len__` (how many examples) and `__getitem__` (return one example by index). In `__init__`, we load the tokenizer from the saved `tokenizer.json` file and load the CodeXGLUE dataset for the requested split:

```
class CodeSummarizationDataset(Dataset):
    def __init__(self, split, tokenizer_path, max_length=128):
        self.tokenizer = Tokenizer.from_file(tokenizer_path)
        self.max_length = max_length
        dataset = load_dataset(
            "google/code_x_glue_ct_code_to_text", "python"
        )
        self.data = dataset[split]
```

The tokenizer and dataset are loaded once in `__init__` rather than in `__getitem__` because `__init__` runs once when we create the object, while `__getitem__` runs for every single example (251,820 times for the training split). Loading them every time would be extremely wasteful.

We set `max_length=128` to truncate sequences that are too long. This is a tradeoff: the Transformer's self-attention compares every token to every other token, so the cost grows quadratically ($128 \times 128 = 16{,}384$ comparisons versus $512 \times 512 = 262{,}144$ at length 512). Most Python functions in CodeXGLUE fit comfortably within 128 tokens, and the ones that get truncated usually still have enough context (the function signature and first few lines) for the model to generate a reasonable docstring.

5.3. Preparing Each Example

When the DataLoader asks for example i, `__getitem__` grabs the code and docstring from the dataset, tokenizes both into lists of integer IDs, and truncates them:

```
code_ids = self.tokenizer.encode(code).ids[:self.max_length]
doc_ids  = self.tokenizer.encode(docstring).ids[:self.max_length - 1]
```

The docstring is truncated to `max_length - 1` because we need room to prepend `[SOS]` or append `[EOS]`. We then build two sequences from the same docstring tokens:

```
decoder_input  = [SOS_ID] + doc_ids    # [SOS] + docstring
decoder_target = doc_ids  + [EOS_ID]   # docstring + [EOS]
```

This is called **teacher forcing**: the decoder receives [SOS] followed by the docstring as its input, and the target is the same docstring followed by [EOS]. The idea is that at each time step, the decoder sees the correct previous token and learns to predict the next one. Once it predicts [EOS], it knows to stop generating.
To see why this shift matters, here is actual output from one training example (truncated for readability):

```
decoder_input:  tensor([   2, 7492, 10665, 576, 13642, ...])
decoder_target: tensor([7492, 10665,  576, 13642,  ..., 3])
```

Notice the decoder_input starts with 2 ([SOS]) and the decoder_target ends with 3 ([EOS]). Everything in between is identical, the same docstring tokens just shifted by one position. The model learns: "given [SOS], predict token 7492; given token 7492, predict token 10665; . . . ; given the last docstring token, predict [EOS] to stop."
Each example is returned as a dictionary of three PyTorch tensors: code_ids (the encoder input), decoder_input (what the decoder receives), and decoder_target (what the decoder should predict).

5.4. Batching with Padding

Different functions have different lengths, but the model needs all sequences in a batch to be the same size for matrix operations. The collate_fn function handles this by padding shorter sequences with the [PAD] token (ID=1). For example, a batch of three code sequences might look like this after padding:

```
[10,  20,  30,   1,   1]   # Function 1 (padded with two 1s)
[50,  60,  70,  80,  90]   # Function 2 (no padding needed)
[100, 110,  1,   1,   1]   # Function 3 (padded with three 1s)
```

This function is not called directly. It is passed to PyTorch's DataLoader, which calls it automatically every time it assembles a batch of examples. The padding value 1 corresponds to [PAD], which the model later learns to ignore during training through the loss function's ignore_index parameter.

6. Model Architecture

Our model is an encoder-decoder Transformer built from three components: an embedding layer, the Transformer module, and an output projection layer. The encoder reads the Python code and the decoder generates the docstring one token at a time.

6.1. The Embedding Layer (Token ID → Vector)

The first step is to convert token IDs into dense vectors. We use a single shared embedding layer for both the encoder (code) and the decoder (docstring), since both sides use the same BPE vocabulary trained in Section 4:

```
self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_id)
```

This creates a lookup table of size $32{,}000 \times 512$, with one row of 512 numbers for each token in the vocabulary. When a token ID is passed in, the layer returns the corresponding 512-dimensional vector. For example, if the input is a sequence of 50 token IDs, the output is a matrix of shape $(50, 512)$.
We use a shared embedding rather than separate ones for the encoder and decoder because both sides operate on the same vocabulary. Tokens like def, return, or function appear in both Python code and English docstrings, so sharing the embedding allows the model to learn a single, consistent representation for each token.
The padding_idx=pad_id argument tells PyTorch to keep the embedding vector for [PAD] (ID=1) as all zeros and not update it during training. This ensures padding tokens carry no information.

After the lookup, we scale the embeddings by $\sqrt{d\_model} = \sqrt{512} \approx 22.6$, following the original "Attention Is All You Need" paper. Without this scaling, the embedding values would be too small relative to the positional encoding that will be added later, and the model would lose track of the actual token identity.

6.2. Positional Encoding

To a Transformer, `for x in range(5)` looks exactly the same as `range(5) x in for`. Without position information, the model cannot tell them apart. Unlike recurrent networks which process tokens one by one in order, the Transformer sees all tokens at once, so we need to explicitly inject position information into the embeddings.
We use the sinusoidal positional encoding from "Attention Is All You Need". During initialization, we precompute a table of size $5{,}000 \times 512$ (5,000 positions is a safe upper limit; our sequences are at most 128 tokens):

```
pe = torch.zeros(max_len, d_model)
position = torch.arange(0, max_len).float().unsqueeze(1)
div_term = torch.exp(
    torch.arange(0, d_model, 2).float()
    * (-math.log(10000.0) / d_model)
)
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
```

Each row in this table is a unique 512-dimensional vector for one position. The even dimensions use sine and the odd dimensions use cosine, with frequencies that decrease across dimensions. This means nearby positions get similar encodings while distant positions get different ones, giving the model a sense of order and relative distance.
The table is stored as a *buffer* (not a learnable parameter) using `register_buffer`, so it moves to the GPU automatically but is never updated during training. In the forward pass, we simply add the positional encoding to the scaled embedding:

```
x = x + self.pe[:, :x.size(1), :]
```

The result is that each token's vector now encodes both *what* the token is (from the embedding) and *where* it appears in the sequence (from the positional encoding).

6.3. The Transformer Module (Vector =¿ Context-Aware Vector)

The core of the architecture is PyTorch's built-in `nn.Transformer`, which contains both the encoder and decoder stacks:

```
self.transformer = nn.Transformer(
    d_model=512,        # vector dimension
    nhead=8,            # attention heads
    num_encoder_layers=6,
    num_decoder_layers=6,
    dim_feedforward=2048,
    dropout=0.1,
    batch_first=True,
)
```

At this point in `__init__`, we are only creating the structure: the layers, the attention heads, and the weight matrices that will store learned knowledge. No data flows through the model until `forward` is called.
What the encoder does. The encoder takes the code embeddings and runs them through 6 layers of self-attention. Each layer lets every token look at every other token in the code and update its own representation accordingly. For example, in `for x in range(5)`, the token `x` starts as a generic variable

name. After passing through the encoder, its vector is enriched with contextual information: that it is a loop iterator, that the loop runs 5 times, and that it is used inside a `for` statement. The "dumb" embedding vector becomes a "smart" context-aware vector.
Each layer uses 8 attention heads, meaning the 512-dimensional vector is split into 8 independent groups of 64 dimensions. Each head can focus on a different type of relationship (one head might track variable names, another might track control flow), and the results are concatenated back together.
What the decoder does. The decoder generates the docstring one token at a time. It also has 6 layers, but each layer has two attention mechanisms: self-attention over the tokens generated so far, and cross-attention over the encoder's output. The cross-attention is what allows the decoder to "look at" the code while writing the docstring.
Masks. Two types of masks are used during the forward pass:

1. **Causal mask** (`target_mask`): A triangular matrix that prevents the decoder from looking at future tokens. When predicting the 3rd word of the docstring, the decoder can only see words 1 and 2, not words 4 or 5. This is generated with:

```
target_mask = self.transformer.generate_square_subsequent_mask(
    target_len)
```

2. **Padding masks**: Boolean masks that tell the attention mechanism to ignore `[PAD]` tokens. Since sequences in a batch are padded to the same length (see Section 5.4), the model needs to know which positions are real tokens and which are just filler:

```
src_key_padding_mask = (code_ids == self.pad_id)
target_key_padding_mask = (decoder_input == self.pad_id)
```

Parameter count. The total model has approximately 77 million trainable parameters: about 16M in the shared embedding, 18M in the 6 encoder layers, 27M in the 6 decoder layers (which have an extra cross-attention block per layer), and 16M in the output projection.

6.4. The Output Layer (Context-Aware Vector =¿ Vocabulary Scores)

The final layer is a linear projection that converts the 512-dimensional output vectors from the decoder into scores over the full vocabulary:

```
self.fc_out = nn.Linear(d_model, vocab_size)
```

Mathematically, this computes $y = xW + b$, where x is a smart vector of 512 numbers, W is a weight matrix of size $512 \times 32{,}000$, and y is a list of 32,000 scores, one for each token in the vocabulary. The token with the highest score is the model's prediction for the next word.
During training, these raw scores (called logits) are passed to the cross-entropy loss function, which converts them to probabilities and measures how far the prediction is from the correct answer. During inference, we take the highest-scoring token at each step (greedy decoding) to generate the docstring.

6.5. Hyperparameter Summary

Table 1 summarizes the model hyperparameters:

7. Training Procedure

Training was conducted on Apple Silicon using MPS (Metal Performance Shaders) acceleration. To ensure compatibility with the Apple Silicon MPS backend, the `PYTORCH_ENABLE_MPS_FALLBACK` environment variable was utilized. This allowed the model to handle specialized Transformer operations by falling back to the CPU where necessary, so training ran without errors on macOS. The full training run completed 10 epochs over the 251,820 training examples.

Hyperparameter	Value
Vocabulary size	32,000
Embedding dimension (d_model)	512
Attention heads	8
Encoder layers	6
Decoder layers	6
Feedforward dimension	2,048
Dropout	0.1
Max sequence length	128 tokens

Table 1: Model hyperparameters.

7.1. Optimizer and Learning Rate Schedule

We use the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$, and $\epsilon = 10^{-9}$, following the original Transformer paper. The base learning rate is set to 1.0 because the actual learning rate is controlled entirely by the Noam schedule:

$$lr = d\_model^{-0.5} \cdot \min(step^{-0.5},\ step \cdot warmup\_steps^{-1.5})$$

This schedule linearly increases the learning rate during the first 4,000 warmup steps (reaching a peak of approximately 7×10^{-4}), then decays proportionally to $1/\sqrt{step}$. The warmup prevents early instability when the model's weights are still random, and the decay allows finer adjustments as the model converges.

7.2. Loss Function

We use cross-entropy loss with `ignore_index=PAD_ID`, so padding tokens do not contribute to the loss. At each position in the output sequence, the loss measures how far the model's predicted probability distribution (over 32,000 tokens) is from the correct next token. Gradient clipping is applied at a maximum norm of 1.0 to prevent exploding gradients.

7.3. Training Configuration

Parameter	Value
Epochs	10
Batch size	32
Batches per epoch	7,870
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$)
Learning rate schedule	Noam (warmup = 4,000 steps)
Gradient clipping	1.0
Loss function	CrossEntropyLoss (ignore_index=1)
Trainable parameters	76,940,544

Table 2: Training configuration.

7.4. Training Results

Table 3 shows the loss and perplexity for each epoch. The best model checkpoint was saved after every epoch that improved validation loss.

Epoch	Train Loss	Train PPL	Val Loss	Val PPL
1	4.8517	127.95	4.2824	72.41
2	3.9161	50.21	4.0021	54.71
3	3.6452	38.29	3.8445	46.74
4	3.5050	33.28	3.7869	44.12
5	3.4040	30.08	3.7200	41.26
6	3.3240	27.77	3.6703	39.26
7	3.2567	25.96	3.6627	38.97
8	3.2006	24.55	3.6049	36.78
9	3.1559	23.48	3.5751	35.70
10	3.1155	22.54	3.5688	35.47

Table 3: Training and validation loss and perplexity (PPL) per epoch.

The largest improvement occurred between epochs 1 and 2, where the validation loss dropped from 4.28 to 4.00. The model continued improving through all 10 epochs, with the best checkpoint saved at epoch 10 (validation loss = 3.5688, perplexity = 35.47). The validation loss decreased every epoch, showing that the model did not overfit despite having 77 million parameters. The gap between training and validation loss widened slightly over time (from 0.57 in epoch 1 to 0.45 in epoch 10 for the absolute difference), which is normal and expected behavior.

8. Inference and Decoding

At inference time, the model generates a docstring for unseen code using **greedy decoding**. Unlike training where the decoder receives the correct previous token (teacher forcing), at inference the decoder must rely entirely on its own predictions. The process works as follows:

1. The source code is tokenized and passed through the encoder, producing context-aware representations.
2. The decoder starts with a single `[SOS]` token.
3. At each step, the model outputs a probability distribution over the 32,000-token vocabulary. The token with the highest score is selected and appended to the decoder input.
4. This repeats until the model produces `[EOS]` or reaches the maximum length of 128 tokens.

8.1. Observed Limitations

Greedy decoding selects only the single most probable token at each step, which can cause **repetition loops**. For example, given a short input like `def add(x, y): return x + y`, the model produces:

```
Summary: return return x, y, return x, return x
```

This happens because the model has learned that tokens like `return`, `x`, and `y` are highly relevant to the input, but greedy decoding gets stuck re-selecting them without producing a coherent sentence. The model performs better on longer, more realistic functions similar to those in the training set, where there is more context for the encoder to work with.

8.2. Beam Search

To address the repetition problem, we replaced greedy decoding with **beam search** (beam width = 5). Instead of committing to a single token at each step, beam search maintains the top 5 most likely sequences simultaneously.

At each decoding step, every active beam is expanded by considering its top 5 next tokens, producing up to 25 candidate sequences. These candidates are ranked by their **cumulative log probability** (the sum of log probabilities of all tokens in the sequence), and only the top 5 are kept. Decoding stops when the highest-scoring beam ends with `[EOS]`.

```
# each beam: (list of token ids, cumulative log probability)
beams = [([SOS_ID], 0.0)]

for each step:
    for each (seq, score) in beams:
        log_probs = log_softmax(model(code, seq))
        top_5 = log_probs.topk(5)
        for token, log_p in top_5:
            candidates.append((seq + [token], score + log_p))
    beams = top 5 candidates by score
```

This avoids the repetition loop because even if "return" scores highest at one position, the cumulative score of a sequence that repeats it multiple times will be lower than a sequence that produces a coherent sentence. The beam search explores multiple paths and selects the one with the best overall probability.

8.3. File-Level Summarization

The `summarize.py` script also supports whole-file summarization via the `--file` flag:

```
python summarize.py --file utils/inference.py
```

Rather than feeding the entire file to the model at once, the script uses Python's built-in `ast` module to parse the source and extract each `FunctionDef`, `AsyncFunctionDef`, and `ClassDef` node individually. For each definition, the corresponding source lines are sliced out and passed as a standalone snippet to the model:

```
tree = ast.parse(source)
for node in ast.walk(tree):
    if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.
       ClassDef)):
        snippet = "\n".join(lines[node.lineno - 1 : node.end_lineno])
        summary = greedy_decode(model, tokenizer, snippet, device=device)
```

This design is deliberate: the model was trained on individual function-level examples from CodeXGLUE, so feeding it one function at a time keeps the input distribution consistent with training. Passing the entire file as a single input would exceed the 128-token context window for any non-trivial file, and the model has no mechanism to relate cross-function context anyway.
The summaries are then printed as a numbered list, giving an overview of what the file does step by step:

```
What 'utils/inference.py' does:

  Step 1 (get_device): return device info
  Step 2 (load_model): Load a trained transformer from a checkpoint file
  Step 3 (load_tokenizer): Load the BPE tokenizer and attach a decoder
  Step 4 (greedy_decode): Generate a docstring using beam search
  Step 5 (beam_search_decode): Maintain the top N sequences at each
     decoding step
```

9. Evaluation Metrics

We evaluate the trained model on the held-out test set (14,918 examples) using four metrics that cover different aspects of summarization quality.

9.1. Test Loss and Perplexity

We compute the same cross-entropy loss used during training, but on the test set. **Perplexity** is defined as the exponential of the average cross-entropy loss:

$$\text{PPL} = e^{\mathcal{L}}$$

where $\mathcal{L}$ is the average per-token cross-entropy loss. Perplexity can be interpreted as the effective number of tokens the model is "choosing between" at each step. A perplexity of 32 means the model is, on average, as uncertain as if it were picking uniformly among 32 equally likely tokens. Lower perplexity indicates a more confident and accurate model.

9.2. BLEU Score

BLEU (Bilingual Evaluation Understudy) [6] measures the overlap of n-grams between the predicted summary and the reference docstring. The score combines precision across n-grams of length 1 through 4, with a brevity penalty to discourage overly short outputs:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{4} \frac{1}{4} \log p_n\right)$$

where p_n is the precision of n-grams of length n, and BP is the brevity penalty. BLEU ranges from 0 to 1, with higher scores indicating better overlap. We use the `nltk.translate.bleu_score` implementation with smoothing to handle cases where higher-order n-gram counts are zero.

9.3. ROUGE Scores

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) [7] complements BLEU by measuring recall in addition to precision. We report three variants:

- **ROUGE-1**: Overlap of unigrams (individual words) between the predicted and reference summaries.
- **ROUGE-2**: Overlap of bigrams (consecutive word pairs), capturing phrase-level similarity.
- **ROUGE-L**: Longest common subsequence (LCS) between the predicted and reference, measuring sentence-level structural similarity without requiring consecutive matches.

All ROUGE scores are reported as F1 scores (harmonic mean of precision and recall), ranging from 0 to 1.

10. Results

10.1. Quantitative Results

Table 4 presents the evaluation metrics computed on the full test set. Loss and perplexity are computed over all 14,918 test examples, while BLEU and ROUGE are computed on a random subset of 500 examples using greedy decoding.

The test perplexity (31.74) is slightly better than the validation perplexity at epoch 10 (35.47), showing the model generalizes well to unseen data. The BLEU score of 15.85% and ROUGE-1 of 48.31% are reasonable for a model trained from scratch without any pretrained language model weights.

10.2. Qualitative Examples

Table 5 shows five examples from the test set comparing the reference docstrings to the model's predictions. The model captures the general topic of each function. For `print_log`, the prediction "Log a message to a logger" is semantically close to the reference "Print a log message to standard error," correctly identifying the logging purpose though missing the specific destination. For the download functions, the model recognizes the URL-related nature but confuses "download" with "search."

Metric	Value
Test Loss	3.4577
Test Perplexity	31.74
BLEU	0.1585
ROUGE-1 F1	0.4831
ROUGE-2 F1	0.1991
ROUGE-L F1	0.3814

Table 4: Evaluation metrics on the test set.

Function	Reference	Predicted
`sina_xml_to_url_list`	Convert XML to URL List. From Biligrab.	Parse URL to URL
`dailymotion_download`	Downloads Dailymotion videos by URL.	Search URL by pattern
`sina_download`	Downloads Sina videos by URL.	Search URL by url
`sprint`	Format text with color or other effects into ANSI escaped string.	Convert text into unicode string.
`print_log`	Print a log message to standard error.	Log a message to a logger.

Table 5: Qualitative comparison of reference and predicted docstrings.

11. Discussion and Limitations

11.1. Strengths

The model converges steadily over 10 epochs without overfitting, shown by the decreasing validation loss across all epochs. Test perplexity improves over the validation perplexity, suggesting good generalization. The ROUGE-1 F1 of 48.31% means nearly half of the unigrams in the reference appear in the predictions, so the model has learned the correct vocabulary for describing code.

11.2. Limitations

Specificity. The model tends to produce generic summaries. For example, it predicts "Search URL by pattern" instead of "Downloads Dailymotion videos by URL." It captures the topic (URLs) but misses the specific action (downloading) and the specific service (Dailymotion).

Short context window. With a maximum sequence length of 128 tokens, longer functions are truncated. The model only sees the function signature and the first few lines, which may not contain enough information for an accurate summary.

Greedy decoding. As discussed in Section 8, greedy decoding can produce repetitive outputs for short or atypical inputs. Beam search (Section 8.2) mitigates this but was not used for the evaluation metrics.

No pretrained weights. Training from scratch with 77M parameters on 252K examples is data-limited compared to pretrained models like CodeT5 or CodeBERT, which use billions of tokens during pretraining. This accounts for much of the gap in specificity and fluency.

Sensitivity to existing docstrings in the input. When the `--file` mode in `summarize.py` is used on functions that already contain docstrings, the model produces better summaries. This is because the encoder receives the existing natural language description as part of its input and effectively paraphrases it. When the docstrings are removed, the model must rely entirely on the code structure and produces weaker, more generic outputs. This reveals that a portion of the model's apparent summarization quality on well-documented code comes from re-reading existing documentation rather than true code understanding. This limitation is masked during standard evaluation since the CodeXGLUE test set does not include

pre-existing docstrings in the `code` field.

11.3. Possible Improvements

A few directions worth exploring: (1) increasing the training data or augmenting it with additional Python repositories, (2) using beam search during evaluation, (3) trying longer sequence lengths (256 or 512 tokens) at the cost of increased training time, and (4) applying label smoothing to the cross-entropy loss to improve generalization.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, et al., "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," *arXiv preprint arXiv:2102.04664*, 2021. `https://github.com/microsoft/CodeXGLUE`

[3] H. Husain, H. H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," *arXiv preprint arXiv:1909.09436*, 2019.

[4] A. Karpathy, "Let's build the GPT Tokenizer," 2024. `https://www.youtube.com/watch?v=MlDP2BVWjS0`

[5] Hugging Face, "Tokenizer Documentation," `https://huggingface.co/docs/transformers/en/main_classes/tokenizer`

[6] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," *Proceedings of the 40th Annual Meeting of the ACL*, pp. 311–318, 2002.

[7] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," *Text Summarization Branches Out*, pp. 74–81, 2004.

Appendix: Group Members and Contributions

Name	Matricola	Contributions
Muhammad Sohail	7178621	Model design and implementation, training pipeline, tokenization, inference, evaluation, testing, and report metadata
Syed Umair Anwar	7172248	Data understanding, evaluation, layer configuration based on data analysis, testing, and report writing